

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	S. BHUVANESH	Reg. No.:	192521004
Section / Batch:	2025	Date:	26 – 08 - 2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Q5. Seating Arrangement (Constraint Satisfaction / Backtracking Problem)

Guests must be seated at a round table such that certain pairs cannot sit together while some must sit adjacent. Clearly define all seating constraints and relationships. Analyze and prioritize constraints to reduce invalid arrangements. Develop a backtracking-based solution to generate valid seating plans. Apply constraint validation effectively during the process. Justify how your solution ensures correctness and completeness.

Code of Conduct

I S.BHUVANESH (192521004) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Bhuvanesh

CO5 – AT1: Constraint-Based Problem Solving

Q5. Seating Arrangement Using Constraint Satisfaction and Backtracking

1. INTRODUCTION

The Seating Arrangement Problem is a Constraint Satisfaction Problem (CSP) in which a group of guests must be arranged around a round table while satisfying several restrictions.

Some guests must sit adjacent, while some pairs cannot sit together. Since many possible arrangements may exist, checking every arrangement directly can be inefficient.

A Backtracking algorithm can solve this problem by constructing the seating arrangement step by step. Whenever a partial arrangement violates any constraint, the algorithm immediately rejects it and goes back to try another possibility.

2. PROBLEM STATEMENT

Consider a round table with 6 guests:

A, B, C, D, E, F

The following constraints are given.

Must-Sit-Adjacent Constraints

1. A must sit next to B.
2. C must sit next to D.

Cannot-Sit-Together Constraints

3. B cannot sit next to C.
4. D cannot sit next to E.
5. E cannot sit next to F.

Additional Constraint

6. Because the table is round, the first and last positions are also considered adjacent.

3. CONSTRAINT REPRESENTATION

The problem can be represented as:

Guests

$$G = \{A, B, C, D, E, F\}$$

Must-Adjacent Set

$$M = \{(A, B), (C, D)\}$$

Cannot-Adjacent Set

$$N = \{(B, C), (D, E), (E, F)\}$$

The solution must satisfy:

(A, B) adjacent

(C, D) adjacent

and

$(B, C), (D, E), (E, F)$ not adjacent

4. WHY THIS IS A CONSTRAINT SATISFACTION PROBLEM

A CSP consists of:

Variables

The seating positions around the table.

$$X_1, X_2, X_3, X_4, X_5, X_6$$

Domains

Each position can contain one of the six guests:

$$D = \{A, B, C, D, E, F\}$$

Constraints

Rules that determine which assignments are allowed.

For example:

- A and B must be neighbors.
- C and D must be neighbors.
- B and C cannot be neighbors.
- D and E cannot be neighbors.
- E and F cannot be neighbors.

The goal is to assign every guest to exactly one position while satisfying all constraints.

5. IMPORTANT PROPERTY OF A ROUND TABLE

In a linear arrangement:

A B C D E F

A and F are not normally considered adjacent.

But in a round table:

A B C D E F

the arrangement wraps around, so:

- A is adjacent to B and F.
- B is adjacent to A and C.
- C is adjacent to B and D.
- D is adjacent to C and E.
- E is adjacent to D and F.
- F is adjacent to E and A.

Therefore, the validation must check the first-last relationship as well.

6. CONSTRAINT PRIORITIZATION

Not all constraints need to be checked at the same stage.

A good backtracking solution checks the most restrictive constraints first.

Priority 1 – Must-Adjacent Constraints

Check:

- $A \leftrightarrow B$
- $C \leftrightarrow D$

These constraints are highly restrictive because each pair must occupy neighboring positions.

Priority 2 – Cannot-Adjacent Constraints

Check:

- $B \leftrightarrow C$
- $D \leftrightarrow E$
- $E \leftrightarrow F$

These constraints eliminate invalid arrangements.

Priority 3 – Unique Guest Constraint

Each guest must appear exactly once.

Priority 4 – Circular Constraint

Finally, check the relationship between the first and last guests.

7. WHY CONSTRAINT PRIORITIZATION HELPS

If constraints are checked only after creating a complete arrangement, the algorithm may generate many invalid arrangements.

For example, suppose a partial arrangement is:

A B C

A and B satisfy their constraint, but B and C cannot sit together.

Therefore, the arrangement can immediately be rejected:

A B C $\rightarrow$ Invalid

There is no need to place D, E and F.

This process is called constraint propagation/pruning.

It reduces the search space and improves efficiency.

8. BACKTRACKING APPROACH

Backtracking is a systematic trial-and-error technique.

The algorithm:

- Fix one guest to remove rotational duplicates.
- Select an unused guest.
- Add the guest to the current seating arrangement.
- Check whether the partial arrangement satisfies the constraints.
- If valid, continue recursively.
- If invalid, remove the guest.
- Try another guest.
- When all guests are placed, check the complete circular arrangement.
- Store the arrangement if every constraint is satisfied.

9. FIXING ONE GUEST

Because the table is circular, rotating the same arrangement does not create a fundamentally different seating plan.

For example:

A B C D E F

and

B C D E F A

represent the same circular arrangement.

Therefore, we can fix A at position 1.

This reduces the number of arrangements from:

$$6! = 720$$

to:

$$(6 - 1)! = 120$$

This is an important optimization.

10. BACKTRACKING SEARCH

Start with:

A _ _ _ _ _

Since A must sit next to B, we should prioritize placing B next to A.

Possible partial arrangement:

A B _ _ _ _

Now B cannot sit next to C.

Therefore, C cannot immediately follow B.

We can try D:

A B D _ _ _

But C must sit next to D.

So place C next to D:

A B D C _ _

Now we need to place E and F.

However:

- D cannot be next to E.
- E cannot be next to F.

We can test possibilities and reject any branch that violates these constraints.

11. CONSTRAINT VALIDATION

For every newly added guest, check:

Condition 1: Guest uniqueness

The guest must not already be present.

Condition 2: Must-adjacent

If A is placed, B must eventually occupy a neighboring position.

If C is placed, D must eventually occupy a neighboring position.

Condition 3: Cannot-adjacent

Check whether the newly placed guest forms a forbidden pair with the previous guest.

For example:

B C

is invalid because:

$$(B, C) \in N$$

Therefore:

Reject the branch.

Condition 4: Circular validation

After all six guests are placed, check:

$$(X_6, X_1)$$

because the last guest and first guest are adjacent.

12. EXAMPLE OF BACKTRACKING

Consider the partial arrangement:

A B C _ _ _

Check B and C:

(B, C) cannot be adjacent

Therefore:

Invalid $\rightarrow$ Backtrack

Remove C:

A B _ _ _ _

Try another guest:

A B D _ _ _

Now D requires C to be adjacent.

Try:

A B D C _ _

This satisfies:

- A-B adjacent ✓
- C-D adjacent ✓
- B-C not adjacent ✓
- D-E not yet violated ✓

Continue placing E and F and validate the remaining constraints.

This demonstrates how backtracking avoids exploring an invalid branch further.

13. PSEUDOCODE

BACKTRACK(seating):

 if all guests are seated:

 if all circular constraints are satisfied:

 record seating

 return

 choose an unseated guest

 for each possible guest:

 if guest satisfies current constraints:

 add guest to seating

 BACKTRACK(seating)

 remove guest from seating

Constraint Checking

VALID(seating, guest):

 if guest is already used:

 return False

 if guest creates a forbidden adjacent pair:

 return False

 if guest violates a required adjacency:

 return False

 return True

14. ALGORITHM

Step 1: Store all guests.

Step 2: Store must-adjacent pairs.

Step 3: Store cannot-adjacent pairs.

Step 4: Fix guest A at the first position.

Step 5: Choose an unseated guest.

Step 6: Check constraints immediately.

Step 7: If the placement is valid, recursively continue.

Step 8: If invalid, backtrack and try another guest.

Step 9: When all guests are placed, check the circular first-last relationship.

Step 10: Output every valid seating arrangement.

15. CORRECTNESS OF THE SOLUTION

The backtracking algorithm is correct because it considers every possible seating choice, except branches that have already been proven invalid.

Case 1: Valid Partial Arrangement

If the current arrangement satisfies all constraints, the algorithm continues adding guests.

Case 2: Invalid Partial Arrangement

If a constraint is violated, that branch cannot produce a valid final solution. Therefore, it is safe to discard the branch.

Case 3: Complete Arrangement

When all guests have been placed, the algorithm checks all constraints, including the circular first-last relationship.

Only arrangements satisfying every constraint are returned.

Therefore:

Every generated solution is valid

and because every potentially valid branch is explored:

No valid solution is missed

Hence, the algorithm is both correct and complete.

16. TIME COMPLEXITY

Without pruning, fixing one guest gives:

$$(n - 1)!$$

possible circular arrangements.

For $n = 6$:

$$(6 - 1)! = 5! = 120$$

In the worst case, the backtracking algorithm can therefore require approximately:

$$O((n - 1)!)$$

time.

Constraint checking can add additional overhead depending on implementation.

However, pruning significantly reduces the practical search space.

17. SPACE COMPLEXITY

The recursion stores the current seating arrangement and information about used guests.

Therefore, auxiliary space is approximately:

$$O(n)$$

for the recursion and current arrangement.

If all valid solutions are stored, additional space is required depending on the number of solutions.

18. ADVANTAGES OF BACKTRACKING

- Systematic: Explores possible arrangements in an organized way.
- Complete: Can find all valid solutions.
- Pruning: Eliminates invalid arrangements early.
- Flexible: Can handle multiple types of constraints.
- Easy to modify: New seating rules can be added to the validation function.

19. LIMITATIONS

- Worst-case time complexity is exponential/factorial.
- Large numbers of guests can produce a very large search space.
- Poor constraint ordering can cause unnecessary exploration.
- Storing all solutions can require significant memory.

20. CONSTRAINT RELATIONSHIP TABLE

Pair	Relationship	Requirement
A – B	Must be adjacent	✓ Required
C – D	Must be adjacent	✓ Required
B – C	Cannot be adjacent	✗ Forbidden
D – E	Cannot be adjacent	✗ Forbidden
E – F	Cannot be adjacent	✗ Forbidden

This table provides a clear representation of the problem constraints.

21. COMPARISON: BRUTE FORCE VS BACKTRACKING

Feature	Brute Force	Backtracking
Search	Generates all arrangements	Builds arrangements step-by-step
Constraint checking	Usually at the end	During construction
Invalid branches	Still generated	Pruned early
Efficiency	Lower	Higher
Complexity	$O((n-1)!)$	Worst case $O((n-1)!)$, but pruning helps
Suitable for CSP	Less suitable	Highly suitable

22. REAL-WORLD APPLICATIONS

The same constraint-based approach can be applied to:

- Wedding seating arrangements
- Conference seating
- Classroom seating
- Examination seating
- Team arrangement
- Resource allocation
- Timetable generation
- Employee scheduling

For example, in an examination hall, students who should not sit together can be represented as cannot-adjacent constraints.

23. CONCLUSION

The Seating Arrangement problem demonstrates how Constraint Satisfaction and Backtracking can be used to solve complex arrangement problems.

The problem is modeled using variables, domains, and constraints. Must-adjacent relationships such as A-B and C-D and cannot-adjacent relationships such as B-C, D-E, and E-F are checked during the construction of the seating arrangement.

Backtracking improves efficiency by rejecting invalid partial arrangements immediately rather than generating complete invalid arrangements. Fixing one guest also reduces rotational duplicates.

Although the worst-case complexity is:

$$O((n-1)!)$$

effective constraint ordering, early validation, and pruning make the approach practical for small and medium-sized seating problems.

Therefore, Backtracking provides a correct, complete, and flexible method for generating valid seating plans under multiple constraints.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____